



PREMIER UNIVERSITY CHATTOGRAM

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Assignment

Course Name	Operating Systems
Course code	CSE 3733
Assignment no	01
Assignment title	Assignment against CT-2 and CT-3
Date of submission	26-02-2025

SUBMITTED TO

SHATABDI ROY MOON

Lecturer

Department of Computer Science and Engineering

REMARKS

SUBMITTED BY

NAME

Sourav Dev

ID

0222220005101033

SEMESTER

5th

BATCH

42nd

SESSION

Fall 2024

SECTION

A

Real-Time Operating System (RTOS) for a Drone Fleet Managing Autonomous Flight:-

Introduction:-

Autonomous drone fleets are becoming an essential tool in various industries, including agriculture, logistics, surveillance, and military operations. For these fleets to function effectively, an efficient and responsive operating system is crucial. A Real-Time Operating System (RTOS) is specifically designed to meet the stringent time constraints required for controlling drones in real-time environments. The RTOS ensures that tasks are executed with minimal latency, providing precise control over drone operations such as flight stability, navigation, obstacle avoidance, communication, and energy management. RTOS for a Drone Fleet is designed to provide a lightweight yet robust system that can handle critical tasks while maintaining real-time responsiveness and efficient resource management. This RTOS enables drones to perform autonomous tasks like coordinated flying, obstacle detection, and power optimization without the need for constant human intervention.

Objective:-

The primary objectives of this RTOS for the drone fleet are as follows:

- **Real-Time Performance:** To ensure timely execution of critical tasks such as flight control, obstacle avoidance, and communication between drones in the fleet.
- **Efficient Task Scheduling:** To optimize the scheduling of tasks, ensuring high-priority tasks are executed promptly while minimizing resource usage for non-essential tasks.
- **Security:** To implement robust security measures that protect drone communications and prevent unauthorized access or hijacking of the system.
- **Power Efficiency:** To optimize the use of battery resources by managing power consumption for both the drone's hardware and system components.
- **Fault Tolerance:** To handle failure scenarios, such as low battery or loss of communication, ensuring the drones can safely recover or return to the base station.

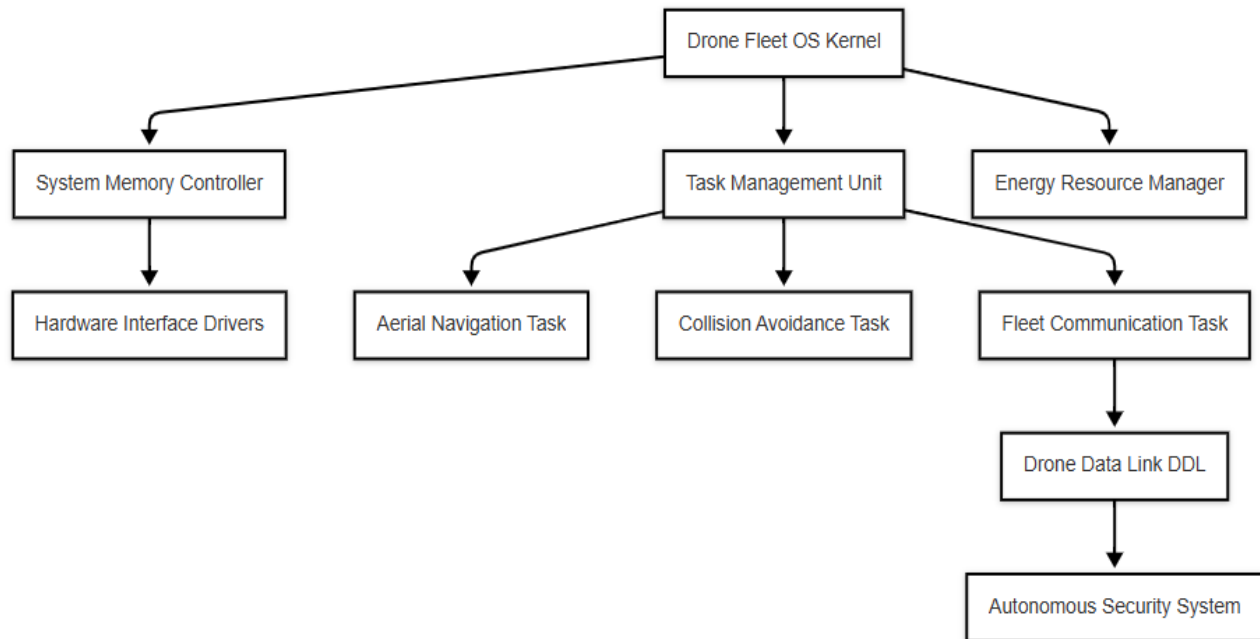
- **Fleet Coordination:** To provide seamless communication and coordination between drones in the fleet, ensuring they work together autonomously to complete missions.

System Architecture:-

The RTOS for a drone fleet consists of several key layers and modules that work together to ensure efficient and reliable operation. Below is an overview of the system architecture:

- **Drone Fleet**
The core component that manages system resources, scheduling, and task execution. The kernel ensures that critical tasks are given priority and executed within strict time constraints.
- **System Memory Controller**
Manages the allocation and deallocation of memory resources. It ensures that memory is used efficiently and tasks don't cause memory conflicts or overload.
- **Task Management Unit**
This module is responsible for scheduling tasks based on priority. Critical tasks such as flight control and collision avoidance are given higher priority than non-critical tasks like logging and monitoring.
- **Hardware Interface Drivers**
Provides the interface between the RTOS and the drone's hardware, such as GPS modules, cameras, sensors, and motors. It enables the system to interact with the physical components of the drone.
- **Energy Resource Manager**
Monitors the drone's power consumption and optimizes the use of battery resources. It ensures that energy is used efficiently to extend the drone's flight time.
- **Fleet Communication Task**
Manages the communication between drones in the fleet and between the drones and the base station. This ensures real-time coordination and data sharing.
- **Drone Data Link (DDL)**
Ensures secure and reliable data transfer between the drones and the base station, supporting telemetry and mission data exchange.
- **Security Module**
Protects the system from unauthorized access and ensures secure communication. It includes encryption, authentication, and monitoring for potential cyber threats.

Diagram:-



Explanation of each component in the lightweight Real-Time Operating System (RTOS) for a drone fleet:

1. Drone Fleet OS Kernel

The core component of the system, responsible for managing resources, controlling tasks, and ensuring synchronization across all components.

2. System Memory Controller

Manages memory allocation and ensures efficient use of available resources for all system components, ensuring smooth task execution.

- **Hardware Interface Drivers:** Responsible for connecting the OS with the drone's hardware, including sensors (GPS, LiDAR, etc.) and actuators (motors, cameras).

3. Task Management Unit

Manages the scheduling and execution of tasks within the system, ensuring that critical tasks receive priority execution.

- **Aerial Navigation Task:** Handles the drone's flight control, ensuring stability, maneuvering, and navigation.
- **Collision Avoidance Task:** Monitors sensor data for obstacles and adjusts the flight path to prevent collisions.

4. Energy Resource Manager

Monitors and manages the drone's power usage to optimize battery life, ensuring tasks are executed efficiently without draining resources prematurely.

5. Fleet Communication Task

Handles communication within the drone fleet and between drones and the base station, ensuring data is exchanged efficiently and securely.

- **Drone Data Link (DDL):** Facilitates the communication link between drones and the base station, supporting the transfer of telemetry and mission data.
- **Autonomous Security System:** Secures communication channels and ensures that only authorized users can interact with the drone system, preventing security breaches.

Functional Modules:-

The RTOS for a drone fleet is divided into several functional modules that handle different aspects of the system:

- **Flight Control Module**
This module ensures real-time flight stabilization and control using data from gyroscopes, accelerometers, and GPS. It dynamically adjusts the drone's speed and direction based on environmental conditions to maintain stability.
- **Obstacle Avoidance Module**
This module processes data from sensors like LiDAR, ultrasonic, and cameras to detect obstacles in the drone's path. It uses algorithms (e.g., A* for path planning) to calculate safe routes and prevent collisions.
- **Communication Module**
Handles communication between the drone fleet and the ground station. This includes transmitting telemetry data such as GPS location, battery status, and flight parameters, as well as receiving commands from the base station.
- **Energy Resource Management Module**
Monitors the battery levels and optimizes power consumption to maximize the drone's operational time. It ensures that tasks like flight control take precedence over less critical tasks to prevent power loss.
- **Security Module**
Provides encryption for communication channels to prevent unauthorized access, verifies the authenticity of commands received by the drone, and monitors the system for potential security breaches or cyber-attacks.
- **Inter-Process Communication (IPC) Module**
This module facilitates data exchange between different system components, such as the flight control, obstacle avoidance, and communication modules, ensuring smooth coordination and data flow across tasks.

Conclusion:-

The RTOS for a Drone Fleet is designed to handle complex tasks, manage real-time sensor data, ensure safety, and optimize power consumption. It guarantees precise control over the drone's operations, coordination within the fleet, and secure communication between drones and the base station, ensuring that the fleet operates efficiently and autonomously in dynamic environments.

Embedded Operating System for Smart Home Device:

Introduction:-

An Embedded Operating System (OS) for a smart home device is designed to provide efficient, reliable, and responsive control over various connected devices in a home. These devices, ranging from smart thermostats and lights to security cameras and voice assistants, require an OS that can handle real-time tasks with minimal resource usage. Smart home devices typically operate with limited processing power, memory, and storage, which necessitates an efficient and lightweight OS that prioritizes performance and power efficiency. An embedded OS for smart home devices must be able to manage tasks like sensor data processing, user interactions, communication with other devices, and power management, all while ensuring security and reliability. Given the wide variety of tasks and device capabilities, this OS must ensure seamless integration with the broader ecosystem of home automation and provide responsive, real-time operations.

Objective:-

The primary objectives of an Embedded Operating System for Smart Home Devices are:

- **Efficient Resource Management:** Optimize CPU, memory, and storage usage to ensure smooth operations on resource-constrained devices.
- **Real-Time Operation:** Provide real-time responses to sensor inputs, user commands, and events, ensuring the system remains responsive.
- **Power Efficiency:** Ensure the system consumes minimal power, enabling long battery life for battery-powered devices and reducing power consumption for wired devices.

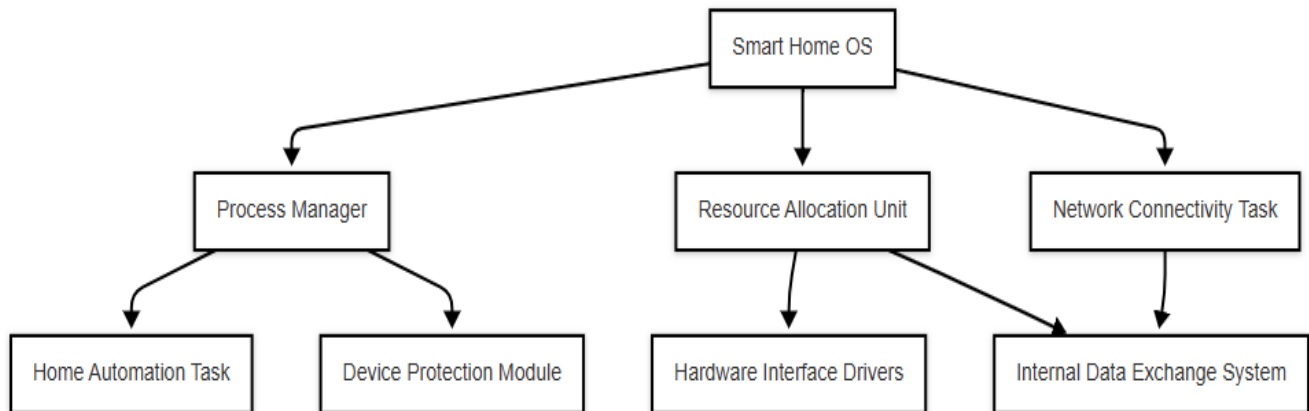
- **Security and Privacy:** Implement robust security features to protect user data, ensure secure communication between devices, and prevent unauthorized access.
- **Connectivity:** Enable seamless communication between smart devices within the home, including support for standards like Wi-Fi, Bluetooth, and Zigbee.
- **User Interaction:** Provide interfaces for users to interact with the device, whether via mobile apps, voice commands, or direct interaction.

System Architecture:-

The architecture of the Embedded Operating System for a Smart Home Device is designed to provide a responsive and efficient control system for smart devices. The components work together to manage resources, handle real-time events, ensure security, and optimize power usage. Below is an overview of the system architecture:

- **Core Layer (Kernel)**
The kernel is the heart of the system and manages all system-level operations such as task scheduling, interrupt handling, and resource management.
- **Resource Management Layer**
This layer includes the Task Scheduler, Device Drivers, and Power Management. It manages the scheduling of tasks based on priorities, facilitates communication with hardware components, and optimizes energy consumption.
- **Communication Layer**
The communication layer handles the connectivity of the device with other devices in the smart home and external networks. This includes Wi-Fi, Bluetooth, and Zigbee protocols, as well as communication with cloud services.
- **Security Layer**
Ensures the safety and integrity of data and device interactions. This includes encryption, secure communication protocols, and user authentication for controlling the smart home device.
- **Application Layer**
This layer interacts with users directly through interfaces such as mobile apps, voice assistants, or physical interfaces (buttons, touch screens). It also handles device-specific logic for tasks like automation rules or user preferences.

Diagram:-



The Smart Home OS manages all essential operations for smart home devices, coordinating various modules to ensure efficient and secure control of connected systems.

Key Components:

- **Smart Home OS:** The central system managing all resources and tasks.
- **Process Manager:** Responsible for task scheduling and management.
 - **Home Automation Task:** Manages tasks like controlling lights, locks, and appliances based on user input or sensor data.
 - **Device Protection Module:** Ensures device security by implementing encryption, authentication, and access control.
- **Resource Allocation Unit:** Manages memory, CPU, and other resources efficiently to ensure optimal device performance.
 - **Hardware Interface Drivers:** Facilitates communication between the OS and physical sensors or actuators (e.g., temperature sensors, motion detectors).
- **Network Connectivity Task:** Manages communication between the device and other smart home systems or the cloud.
- **Internal Data Exchange System:** Handles the exchange of data between system components, ensuring smooth operation and real-time responses.

Functional Modules:-

- **Sensor Data Processing Module**
This module is responsible for reading data from various sensors (e.g., temperature, motion, humidity) and processing it in real-time. It provides immediate feedback to the system, such as adjusting temperature settings or triggering an alarm based on detected motion.
- **Actuator Control Module**
The actuator control module handles the control of devices like lights, locks, and fans based on the inputs from sensors or user commands. It ensures that actions are taken promptly based on the real-time data received.
- **Communication Module**
This module manages communication with other devices in the smart home and external networks. It ensures that commands and data are transmitted efficiently and securely between devices, smartphones, and cloud-based systems.
- **User Interaction Module**
This module enables user interaction through mobile apps, voice commands (e.g., Alexa, Google Assistant), or physical interfaces (buttons or touchscreens). It processes user inputs and triggers the appropriate actions.
- **Security and Privacy Module**
This module provides encryption and authentication mechanisms to ensure that only authorized users can control the smart device. It also includes security features like intrusion detection and protection from unauthorized remote access.
- **Power Management Module**
The power management module is responsible for minimizing power consumption. It monitors the device's activity level and adjusts power states accordingly. For battery-powered devices, it ensures that the device remains operational for long periods without frequent recharging.

Conclusion:-

An embedded operating system for a smart home Device is crucial for enabling seamless and efficient control over the smart home ecosystem. It ensures that tasks are performed in real time, devices communicate effectively, and security is maintained throughout the system. By managing resources efficiently and ensuring low power consumption, the system can function autonomously while offering users control and security. This makes it an essential component of modern smart home solutions.

Mini OS for a gaming console with limited hardware resources

Introduction:-

An Embedded Operating System (OS) for smart home devices is a lightweight, resource-efficient system designed to control and manage various devices in a smart home environment. These devices include thermostats, lighting systems, security cameras, voice assistants, and other IoT-based devices that require constant monitoring and control. The OS is responsible for ensuring real-time task execution, low power consumption, secure communication, and seamless interaction between devices. Smart home devices often operate with limited resources, so the embedded OS must be optimized for efficiency, reliability, and low latency. It is also crucial that the system provides robust security features to protect user data and prevent unauthorized access. This OS is essential for enabling devices to perform automated tasks, communicate with other devices, and ensure user safety and comfort in the home.

Objective:-

The main objectives of the Embedded Operating System for Smart Home Devices are:

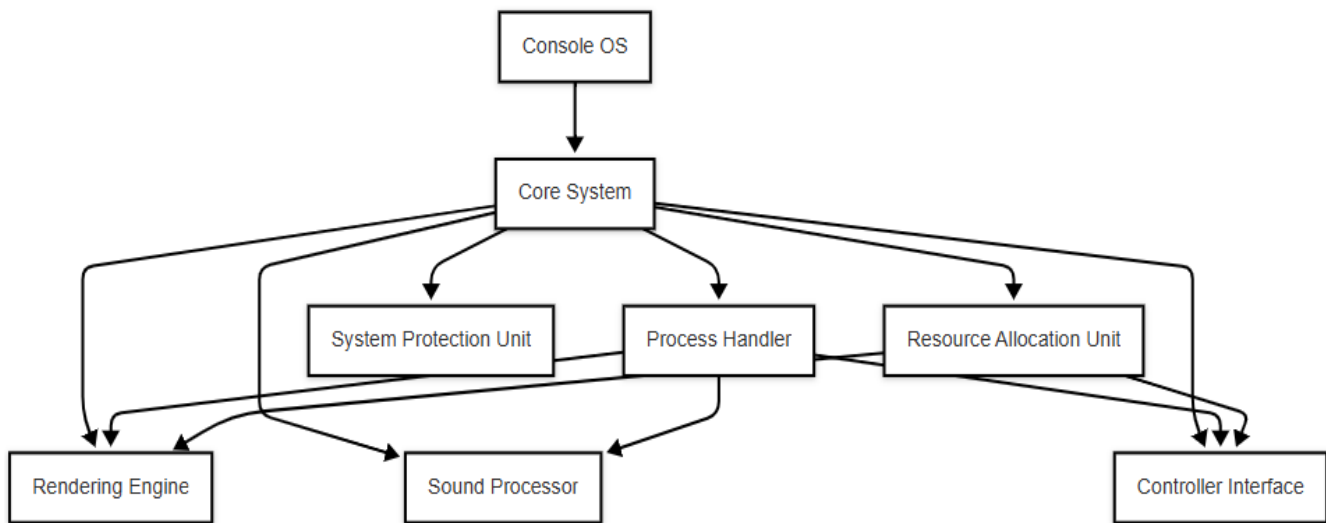
- **Efficient Resource Utilization:** Optimize the usage of CPU, memory, and storage to run multiple tasks efficiently on resource-constrained devices.
- **Real-Time Operation:** Ensure real-time responsiveness to sensor inputs, user commands, and events such as motion detection or voice commands.
- **Low Power Consumption:** Minimize power usage to ensure long operation times for battery-powered devices and reduce energy consumption for wired devices.
- **Security and Privacy:** Implement robust security features to protect communications, prevent unauthorized access, and safeguard user privacy.
- **Seamless Connectivity:** Ensure smooth communication between smart devices, both within the home and with external networks or cloud services.
- **User-Friendly Interaction:** Provide interfaces for users to interact with and control devices through mobile apps, voice assistants, or local interfaces.

System Architecture:-

The architecture of the Embedded Operating System for Smart Home Devices is designed to ensure efficient task management, communication, security, and power management. Below is a breakdown of the system architecture:

- **Core Layer (Smart Home OS)**
 - The central system that manages all components, resources, and task scheduling. It is responsible for prioritizing tasks and ensuring real-time performance.
- **Task Management Layer (Process Manager)**
 - Responsible for scheduling and managing tasks in the system. It ensures that high-priority tasks (e.g., sensor data processing, real-time user commands) are executed promptly.
 - Interacts with various functional modules to ensure smooth operation.
- **Resource Management Layer (Resource Allocation Unit)**
 - Manages memory, CPU, and other resources efficiently. It ensures that each task gets the necessary resources without overloading the system.
- **Hardware Interaction Layer (Hardware Interface Drivers)**
 - Interfaces with sensors, actuators, and other hardware components. These drivers enable the OS to interact with physical devices like temperature sensors, lights, locks, and cameras.
- **Connectivity Layer (Network Connectivity Task)**
 - Manages communication between the device and other smart home devices, as well as external networks like the cloud. It supports communication protocols like Wi-Fi, Zigbee, and Bluetooth.
- **Security Layer (Device Protection Module)**
 - Ensures that data is encrypted and securely transmitted. It protects against unauthorized access and monitors for potential security breaches.
- **Data Exchange Layer (Internal Data Exchange System)**
 - Facilitates efficient communication between different system components, ensuring that data is shared and processed in real time.

Diagram:-



The diagram illustrates the architecture of the Console OS for a gaming console, showing the interaction between the key components of the system.

Key Components and Their Roles:

- **Console OS:**

- The central system that oversees all processes and resources within the console. It ensures that all tasks are executed efficiently and in real-time.

- **Core System:**

- The core component of the Console OS, responsible for managing fundamental tasks, including process scheduling, task execution, and resource allocation. It coordinates the communication between various modules in the system.

- **Process Handler:**

- Responsible for managing and scheduling the execution of tasks. It assigns priorities to tasks like rendering graphics, processing audio, and handling user inputs. Tasks are executed based on their priority, ensuring smooth gaming performance.

- **Resource Allocation Unit:**

- Manages system resources like memory, CPU time, and hardware components. It ensures that these resources are allocated efficiently to

tasks without overloading the system, particularly important in a console with limited resources.

- **System Protection Unit:**

- Provides security for the system by ensuring that processes cannot interfere with one another. This module monitors the system for potential threats or unauthorized access and prevents malicious activities.

- **Rendering Engine:**

- Handles the graphical rendering tasks, ensuring that the game's visuals are processed and displayed correctly. It is directly linked to the Core System, Process Handler, and Resource Allocation Unit as it requires substantial resources for real-time graphics rendering.

- **Sound Processor:**

- Manages audio processing, ensuring that sounds are correctly processed and outputted. It interacts with both the Core System and Process Handler to ensure that the audio matches the gameplay experience and is synchronized with other media.

- **Controller Interface:**

- Manages user input, such as signals from game controllers. It communicates with the Core System and Process Handler, ensuring that user inputs are processed correctly and in real time.

Flow and Interactions:-

- Console OS acts as the overarching control system that ensures all components work together.
- Core System interacts with all other components (such as Resource Allocation Unit, Process Handler, Rendering Engine, etc.) to manage resources and process data efficiently.
- Process Handler directly communicates with components like Rendering Engine, Sound Processor, and Controller Interface to manage tasks related to graphics, audio, and user inputs.
- The System Protection Unit ensures that the system remains secure and that tasks do not interfere with each other, ensuring smooth and safe execution of the OS.
- Resource Allocation Unit is central to managing the console's memory and CPU resources, ensuring tasks like rendering and audio processing get the necessary resources without causing system overload.

Conclusion:-

This architecture ensures the efficient operation of a gaming console, where real-time processing is required for rendering graphics, processing audio, and handling user input. The Console OS coordinates the tasks, protects the system from threats, and optimizes resource usage to ensure smooth gameplay and performance.

Answer to the question no 2

My student ID is 0222220005101033. It is used to perform the FIFO, Optimal and LRU page replacement algorithms using frame size of 3.

Reference String: 0, 2, 2, 2, 2, 2, 0, 0, 0, 5, 1, 0, 1, 0, 3, 3

1. FIFO (First-In-First-Out) Algorithm

FIFO replaces the oldest page in memory when a page fault occurs.

Reference	Frame State (FIFO)	Page Fault?	Notes
0	[0]	Miss	Frame is empty.
2	[0, 2]	Miss	Add to frame.
2	[0, 2]	Hit	Page 2 is already in frame.
2	[0, 2]	Hit	Page 2 is already in frame.
2	[0, 2]	Hit	Page 2 is already in frame.
2	[0, 2]	Hit	Page 2 is already in frame.
0	[0, 2]	Hit	Page 0 is already in frame.
0	[0, 2]	Hit	Page 0 is already in frame.
0	[0, 2]	Hit	Page 0 is already in frame.
5	[0, 2, 5]	Miss	Add to frame.
1	[2, 5, 1]	Miss	Replace oldest page (0).
0	[5, 1, 0]	Miss	Replace oldest page (2).
1	[5, 1, 0]	Hit	Page 1 is already in frame.
0	[5, 1, 0]	Hit	Page 0 is already in frame.
3	[1, 0, 3]	Miss	Replace oldest page (5).
3	[1, 0, 3]	Hit	Page 3 is already in frame.

- Total Page Faults (Misses): 7 and Total Hits: 9
- Hit Ratio: 56.25% and Miss Ratio: 43.75%

2. Optimal Algorithm

Optimal replaces the page that will not be used for the longest period in the future.

Reference	Frame State (Optimal)	Page Fault?	Notes
0	[0]	Miss	Frame is empty.
2	[0, 2]	Miss	Add to frame.
2	[0, 2]	Hit	Page 2 is already in frame.
2	[0, 2]	Hit	Page 2 is already in frame.
2	[0, 2]	Hit	Page 2 is already in frame.
2	[0, 2]	Hit	Page 2 is already in frame.
0	[0, 2]	Hit	Page 0 is already in frame.
0	[0, 2]	Hit	Page 0 is already in frame.
0	[0, 2]	Hit	Page 0 is already in frame.
5	[0, 2, 5]	Miss	Add to frame.
1	[0, 2, 1]	Miss	Replace page 5 (not used soon).
0	[0, 2, 1]	Hit	Page 0 is already in frame.
1	[0, 2, 1]	Hit	Page 1 is already in frame.
0	[0, 2, 1]	Hit	Page 0 is already in frame.
3	[0, 1, 3]	Miss	Replace page 2 (not used soon).
3	[0, 1, 3]	Hit	Page 3 is already in frame.

- Total Page Faults (Misses): 5 and Total Hits: 11
- Hit Ratio: 68.75% and Miss Ratio: 31.25%

3. LRU (Least Recently Used) Algorithm

LRU replaces the page that has not been used for the longest time.

Reference	Frame State (LRU)	Page Fault?	Notes
0	[0]	Miss	Frame is empty.
2	[0, 2]	Miss	Add to frame.
2	[0, 2]	Hit	Page 2 is already in frame.
2	[0, 2]	Hit	Page 2 is already in frame.
2	[0, 2]	Hit	Page 2 is already in frame.
2	[0, 2]	Hit	Page 2 is already in frame.
0	[0, 2]	Hit	Page 0 is already in frame.
0	[0, 2]	Hit	Page 0 is already in frame.
0	[0, 2]	Hit	Page 0 is already in frame.
5	[0, 2, 5]	Miss	Add to frame.
1	[2, 5, 1]	Miss	Replace least recently used (0).
0	[5, 1, 0]	Miss	Replace least recently used (2).
1	[5, 1, 0]	Hit	Page 1 is already in frame.
0	[5, 1, 0]	Hit	Page 0 is already in frame.
3	[1, 0, 3]	Miss	Replace least recently used (5).
3	[1, 0, 3]	Hit	Page 3 is already in frame.

- Total Page Faults (Misses): 6 and Total Hits: 10
- Hit Ratio: 62.5% and Miss Ratio: 37.5%

Comparison of Algorithms:-

Algorithm	Hit Ratio	Miss Ratio
FIFO	56.25%	43.75%
Optimal	68.75%	31.25%
LRU	62.5%	37.5%

- Optimal Algorithm performs the best with the highest Hit Ratio (68.75%) and the lowest Miss Ratio (31.25%).
- LRU performs better than FIFO, but worse than Optimal.
- FIFO has the lowest Hit Ratio (56.25%) and the highest Miss Ratio (43.75%).

